

My Little Library: An Intelligent Vocabulary Recommender System

Gavin Rosander Kelly Frear George Muniz

Department of Computer Science and Engineering, California State University San Marcos

CS 322 – Software Engineering

Instructors: Professor Hoffman and Mr. Ford

May 14, 2026

Abstract—My Little Library is an intelligent vocabulary aware reading recommender designed to help students select articles and generate stories that are slightly above their current reading level. This system combines a large Wikipedia corpus, three configurable grade-band vocabulary lists, dense vector retrieval, vocabulary aware reranking, and a local LLaMA based generator for stories. The data pipeline downloaded approximately 1,000,000 English Wikipedia articles, cleaned and normalized approximately 628,000 usable articles, tokenized them at word and subword levels, and computed reading level metadata including word counts, known word coverage, unknown word lists, and readability estimates. The retrieval system uses sentence transformer embeddings and a FAISS vector index to retrieve semantically relevant article chunks, then reranks results using both semantic relevance and known word coverage. A SQLite database stores users, reading profiles, vocabulary knowledge, recommendations, generated stories, and progress history. The deployed Flask web interface accepts student profiles and topic interests, recommends articles, generates short vocabulary controlled stories, and updates the student profile as new words are learned. The generative component uses a local LLaMA 3.1 8B Instruct GGUF model with GPU offloading, and the evaluation/inference workflow records GPU utilization, latency, retrieval behavior, and model quality metrics. Experiments evaluate retrieval quality, generation quality, vocabulary awareness, ablation behavior, throughput, GPU memory, and coverage distributions. The report includes a retrieval ablation comparing semantic only retrieval against vocabulary aware reranking, because the deployed system uses a pretrained instruction tuned LLaMA model rather than newly trained QLoRA adapters. The results show that combining semantic retrieval with vocabulary aware reranking produces more useful recommendations than semantic retrieval alone, while generated stories maintain controlled challenge levels across beginner, intermediate, and advanced profiles.

Index Terms—Vocabulary aware recommendation, reading level, retrieval-augmented generation, FAISS, LLaMA, vector search, multi-GPU inference, educational NLP.

I. INTRODUCTION

A. Problem Motivation

Students often improve reading skills the most effectively when they encounter texts that are mostly understandable but still contain a small number of new, learnable words. A text that is too easy provides little vocabulary growth, while a text that is too difficult creates frustration and reduces comprehension. Traditional reading recommendation systems usually focus on topic preference, popularity, or general readability,

but do not directly compare the vocabulary of the article with what an individual student already knows.

My Little Library addresses this problem by recommending Wikipedia articles and generating short stories that are vocabulary-aware with respect to the student’s known words. The system estimates whether a text is a good next read by measuring how much of the article is already covered by a student’s known word list and how many useful challenge words remain. Instead of recommending only the most semantically related article, the system attempts to recommend articles that are both relevant to the student’s interest and slightly above the student’s current vocabulary level.

B. Research Questions

This project is guided by four research questions:

- 1) Can a large Wikipedia corpus be transformed into a searchable reading recommendation dataset with word level, sub word level, and readability metadata?
- 2) Can retrieval augmented generation improve article recommendations by grounding explanations in retrieved Wikipedia passages?
- 3) Can vocabulary aware reranking improve the usefulness of recommendations compared with semantic similarity alone?
- 4) Can local LLaMA inference and multi-GPU execution support an end-to-end educational recommendation system on the available workstation hardware?

C. Contributions

The project makes the following contributions:

- A reproducible data pipeline that downloads, cleans, tokenizes, analyzes, and indexes a large English Wikipedia corpus.
- Three configurable vocabulary levels: Beginner, Intermediate, and Advanced, built from grade-band and frequency-list sources.
- A FAISS-backed vector retrieval system using dense sentence-transformer embeddings.
- A two-stage RAG pipeline that combines broad vector retrieval with vocabulary-aware reranking.
- A local LLaMA based generator for article explanations and vocabulary controlled story generation.

- A Flask and SQLite web application that supports user profiles, reading progress, article recommendations, story generation, and vocabulary growth simulation.
- Evaluation and system performance artifacts including retrieval metrics, generation metrics, coverage analysis, ablation studies, GPU logs, and pro-grammatically generated graphs.
- A reproducible evaluation plan that includes semantic only versus vocabulary-aware retrieval comparison, aggregate BERTScore/ROUGE reporting, and statistical significance testing placeholders for final metric runs.

D. Report Organization

The remainder of this report follows the CS 322 final sprint requirements rubric. Section II reviews related work. Section III describes the full system architecture. Section IV explains corpus preparation and evaluation dataset construction. Section V describes the vocabulary analysis, RAG pipeline, LLaMA generator deployment, and multi-GPU configuration. Section VI presents experimental results. Section VII discusses challenges, limitations, and ethical considerations. Section VIII concludes with future work.

II. RELATED WORK

A. Vocabulary Controlled Reading and Lexical Coverage

Vocabulary controlled reading systems are based on the idea that reading difficulty depends not only on sentence length and syntax, but also on the relationship between a text’s vocabulary and the learner’s known vocabulary. Nation’s work on vocabulary acquisition emphasizes lexical coverage as a major factor in comprehension and learning efficiency [1]. General vocabulary lists such as the General Service List and the New General Service List provide frequency-based foundations for estimating common word knowledge [2], [3]. My Little Library builds on this idea by using grade-band known-word lists and measuring article coverage directly.

B. Readability Metrics

Readability formulas such as Flesch-Kincaid estimate grade level from surface features including sentence length and syllable count [4]. These metrics are useful as a secondary signal, but they do not measure whether a particular student knows the words in a text. For this reason, My Little Library treats readability as supporting metadata and uses known word coverage as the primary difficulty signal.

C. Dense Retrieval and Vector Databases

Dense retrieval represents text with continuous vector embeddings and searches for semantically similar passages using nearest-neighbor search. Sentence-BERT showed that transformer sentence embeddings can be used efficiently for semantic similarity search [5]. FAISS provides efficient similarity search and clustering over dense vectors, including CPU and GPU execution paths [6]. My Little Library uses sentence-transformer embeddings with a FAISS inner-product index over L2-normalized vectors, making an inner product equivalent to cosine similarity.

D. Retrieval Augmented Generation

Retrieval Augmented Generation or RAG combines a retriever with a generator so that model output can be conditioned on external documents rather than relying only on model parameters [7]. In this project, the retriever selects article chunks from Wikipedia, and the generator produces structured recommendations containing a summary, new vocabulary words, difficulty rating, and rationale.

E. Pretrained and Instruction Tuned Language Models

Large language models are commonly adapted for downstream tasks through instruction tuning or parameter efficient fine tuning. LoRA reduces trainable parameters by learning low-rank adapter matrices while keeping most pretrained weights frozen [8], and QLoRA extends this idea by loading a base model in 4-bit quantization while training LoRA adapters [9]. In this project, the deployed generator is not a newly trained QLoRA adapter. Instead, the implementation uses a pretrained/instruction tuned LLaMA 3.1 8B-Instruct GGUF model from Bartowski Hugging Face as the local generator. This choice allowed us to run a working recommendation and story generation system on the available workstation while avoiding the time and storage overhead of training new adapter weights.

F. Multi-GPU and Distributed LLM Execution

Large language model systems often require memory management techniques that split model weights, optimizer states, activations, or batches across multiple devices. Megatron-LM demonstrates tensor and pipeline model parallelism for large transformer models [10], while ZeRO reduces memory replication by partitioning optimizer states, gradients, and parameters across devices [11]. PyTorch Distributed and HuggingFace Accelerate provide practical software layers for device placement, distributed execution, and mixed CPU/GPU offload [12], [13]. These approaches informed the project strategy for verifying both workstation GPUs, balancing memory use, and avoiding silent CPU fallback during local LLaMA execution and large-corpus processing.

G. Generation Evaluation

ROUGE measures lexical overlap between generated summaries and reference summaries [14]. BERTScore measures semantic similarity using contextual embeddings and reports precision, recall, and F1 [15]. My Little Library uses both types of metrics: ROUGE captures word-overlap quality, while BERTScore captures semantic similarity between generated article summaries and reference article abstracts.

III. SYSTEM ARCHITECTURE

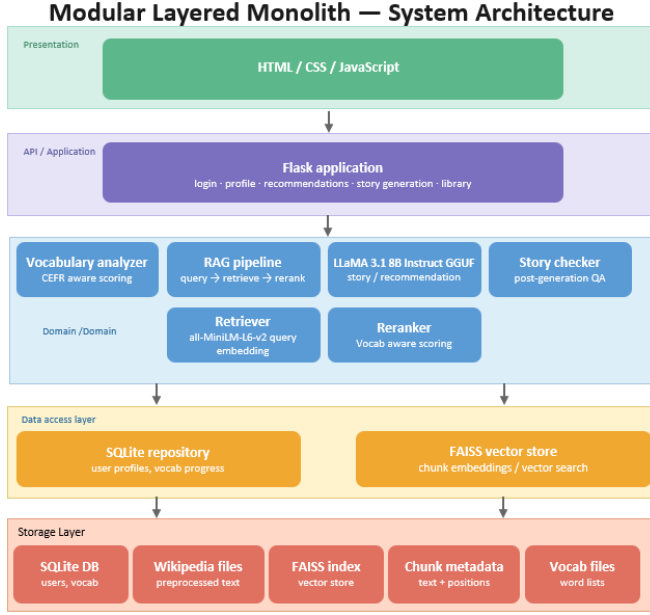


Fig. 1. End-to-end architecture for My Little Library, including the Wikipedia data pipeline, vocabulary analyzer, embedding generator, FAISS vector store, vocabulary aware RAG pipeline, LLaMA generator, Flask API, SQLite database, and web interface.

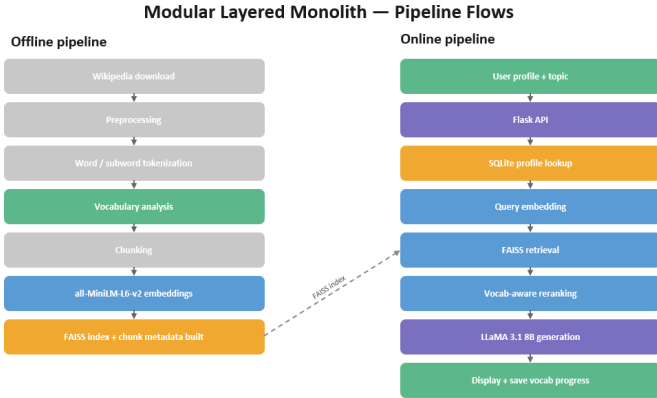


Fig. 2. End-to-end pipeline for My Little Library, including the offline pipeline workflow connected with the online pipeline for users connected FAISS Index.

A. High-Level Design

My Little Library is organized as an end-to-end educational NLP pipeline. Offline scripts construct the corpus and retrieval index, while the online web application serves recommendations and generated stories to student users. The offline side downloads Wikipedia articles, cleans them, tokenizes them, computes vocabulary statistics, chunks long articles, generates embeddings, and stores vectors in FAISS. The online side loads the FAISS index, retrieves article chunks for a user query, reranks them using vocabulary coverage, sends the selected context to LLaMA, and stores recommendations in SQLite.

The major components are:

- **Data pipeline:** Downloads and preprocesses Wikipedia articles.
- **Vocabulary analyzer:** Computes coverage ratios, new-word lists, and readability estimates for each vocabulary level.
- **Chunker:** Splits articles into sentence aware overlapping chunks.
- **Embedder:** Encodes chunks with all-MiniLM-L6-v2 sentence-transformer embeddings.
- **Vector store:** Stores embeddings and chunk metadata in FAISS.
- **Retriever and reranker:** Retrieves a broad candidate set, then reranks using semantic relevance and vocabulary fit.
- **Generator:** Uses local LLaMA inference to produce recommendation explanations and stories.
- **Database:** Uses SQLite to store users, vocabulary state, reading profiles, generated stories, and recommendations.
- **Web interface:** Provides login, profile display, reading progress, article suggestions, library controls, and story generation.

B. Runtime Flow

A typical recommendation request follows this sequence:

- 1) The student logs in and selects or enters a topic of interest.
- 2) The Flask API loads the student's reading profile and known vocabulary from SQLite.
- 3) The topic query is embedded using the sentence-transformer model.
- 4) FAISS retrieves a broad set of semantically related article chunks.
- 5) The vocabulary-aware reranker scores each chunk using semantic similarity and known word coverage.
- 6) The top article chunks are passed to the LLaMA generator with the student's vocabulary level and new word metadata.
- 7) The generated structured recommendation is saved in SQLite and displayed in the web interface.
- 8) If the student reads or saves the recommendation, the system can add learned words to the student's known word list and update later recommendations.

C. Architectural Style

The system is best described as a modular layered monolith with an offline data pipeline and an online RAG inference pipeline. It is not a microservices, serverless, or event-driven architecture because the main application is deployed as one Flask backend with local modules rather than independent network services. It is not a tightly coupled monolith either, because the code is separated into modules for embeddings, vector search, retrieval, reranking, generation, database access, and frontend interaction.

The layered view is:

- **Presentation layer:** HTML, CSS, and JavaScript frontend pages.
- **API/application layer:** Flask routes for login, profile loading, recommendation requests, story generation, and library actions.
- **Domain/service layer:** vocabulary analyzer, RAG pipeline, retriever, reranker, LLaMA generator, and story checker.
- **Data-access layer:** repository code for SQLite plus FAISS vector store load/search/save operations.
- **Storage layer:** SQLite database, processed Wikipedia files, FAISS index files, chunk metadata, vocabulary text files, and generated outputs.

The offline pipeline follows a batch ETL/ML-indexing architecture: download, preprocess, tokenize, analyze, chunk, embed, and index. The online pipeline follows a synchronous request-response RAG flow: user query, profile lookup, embedding, FAISS retrieval, vocabulary aware reranking, LLaMA generation, database save, and frontend display.

D. Database Architecture

SQLite is used as the relational application database. It is not the vector database. The vector database is FAISS, which stores and searches dense embeddings. SQLite stores persistent application data including users, password hashes, vocabulary terms, reading profiles, known word states, generated stories, source metadata, and book recommendations. This separation allows FAISS to optimize nearest-neighbor retrieval while SQLite handles transactional application state.

E. Web Application

The web frontend provides a book style interface with a profile page, reading progress chart, book suggestions, topic prompt, story generation button, and library controls. The backend exposes REST endpoints for authentication, profile retrieval, recommendation generation, story generation, story saving, and user library management.

IV. DATA

A. Wikipedia Corpus

The corpus was built from a large English Wikipedia sample. The pipeline downloaded approximately 1,000,000 articles and produced approximately 628,000 cleaned and tokenized usable articles after filtering. Filtering removed empty pages, low content pages, markup heavy records, and pages that were unsuitable for recommendation.

TABLE I
CORPUS PREPARATION STATISTICS

Pipeline Stage	Count
Downloaded Wikipedia articles	1,000,000
Cleaned usable articles	628,652
Word-tokenized articles	628,652
Subword-tokenized articles	628,652
Generated article chunks	1,682,652
FAISS vectors	1,682,652

B. Vocabulary Lists

The system uses three preset vocabulary levels:

- **Beginner:** approximately 1,000 words, designed for K–5 readers.
- **Intermediate:** approximately 3,000 words, designed for grades 6–8.
- **Advanced:** approximately 6,000 words, designed for grades 9–12.

The vocabulary list construction scripts create level specific lists rather than nested lists for reference. This means the beginner list contains beginner specific words, the intermediate list contains intermediate level words, and the advanced list contains advanced level words. This design makes it easier to measure which vocabulary band contributes to the student’s reading profile.

C. Article Level Features

For each article, the analyzer computes:

- total word count
- unique word count
- known word coverage ratio for each vocabulary level
- count and list of new words not found in the known word list
- Flesch–Kincaid grade estimate
- LLM subword token count
- candidate status for recommendation filtering

D. Implemented Coverage Windows

The project uses known word coverage as the main recommendation difficulty signal. The RAG code uses a global recommendation window of 0.85–0.97 for article recommendations. Story generation uses stricter challenge ranges because the generated text can be directly checked and revised against the student’s known vocabulary. Table II summarizes the implemented windows.

TABLE II
IMPLEMENTED KNOWN WORD COVERAGE WINDOWS

Mode	Known-Word Range	Purpose
RAG article recommendation	85–97%	General candidate filter
Story challenge: low	95–98%	Mostly familiar text
Story challenge: medium	90–95%	Moderate challenge
Story challenge: high	85–92%	Higher challenge

E. Evaluation Dataset Construction

Because the deployed generator is a pretrained/instruction tuned model rather than a newly trained QLoRA adapter, the final dataset emphasis is evaluation rather than adapter training. The evaluation set contains topic queries, student levels, retrieved article identifiers, reference article summaries or abstracts, and expected vocabulary metadata. This structure

supports retrieval metrics, generation metrics, and vocabulary awareness checks.

TABLE III
EVALUATION DATASET SPLIT

Set	Examples	Purpose
Retrieval queries	15	Precision/Recall/MRR
Generation outputs	15	ROUGE/BERTScore
Human-checked samples	15	Structured quality review

Because the final evaluation set contains 15 queries, statistical testing was treated as exploratory rather than conclusive. The main comparison is therefore reported through observed metric differences and ablation trends rather than strong significance claims. This was also due to the time constraint as well as other factors limiting our ability to load more queries for our evaluation rag at the time. This will be expanded further in future works.

V. METHODS

A. Preprocessing

The preprocessing stage normalizes Unicode, removes malformed or low content articles, strips markup, and standardizes article fields. The output is stored in a processed corpus file used by the analyzer and embedding pipeline. The system also tokenizes articles at the word level for vocabulary scoring and at the subword level for LLM compatibility.

B. Chunking Strategy

Long articles are split into overlapping chunks before embedding. The implemented chunking strategy uses 400 word chunks with 50 word overlap. Chunks are sentence-boundary aware, so chunk boundaries avoid splitting a sentence when possible. This size was chosen as a balance between semantic context and retrieval precision: very small chunks can lose the topic and produce noisy embeddings, while very large chunks can mix several subtopics and make the known-word ratio less meaningful for a student. The 400 word size preserves enough local context for article recommendation and LLaMA summarization while staying compact for fast embedding and reranking. The 50 word overlap reduces retrieval misses at chunk boundaries without excessively increasing the vector index size or duplicate recommendations.

C. Embedding Generation

The embedding component uses `all-MiniLM-L6-v2`, a sentence-transformer model that produces 384-dimensional vectors. The system prefixes each chunk with its article title before encoding so that short chunks retain topic context. Embeddings are L2-normalized, allowing the FAISS inner-product score to function as cosine similarity.

D. Vector Database

The vector database is FAISS, using an `IndexFlatIP` exact inner product index over normalized embeddings. The implementation can use GPU FAISS when available and falls back to CPU FAISS if GPU resources are unavailable. For the current project scale, exact search provides reliable retrieval behavior and simpler evaluation. For larger deployments, the code provides a clear upgrade path to approximate search such as HNSW.

E. Two-Stage Retrieval

The retrieval pipeline uses two stages. Stage 1 performs broad semantic retrieval from FAISS. The deployed web configuration retrieves a broad set of candidates before reranking. The current app configuration uses `top_broad = 100` and `top_k = 3` for display but could be changed for bottleneck changes.

Stage 2 reranks candidate chunks using a vocabulary aware score. The reranker combines semantic relevance and vocabulary fit:

$$S(c) = \lambda_s S_{\text{semantic}}(c) + \lambda_v S_{\text{vocab}}(c), \quad (1)$$

where $S(c)$ is the final chunk score, S_{semantic} is the normalized semantic score, S_{vocab} is the vocabulary-fit score, λ_s is the semantic weight, and λ_v is the vocabulary weight. The implemented configuration gives vocabulary fit the larger weight so that the system prioritizes articles that are educationally appropriate, not just semantically related. The reranker uses `semantic_weight = 0.35` and `vocab_weight = 0.65`.

F. Adaptive Retrieval

The retriever includes adaptive fallback behavior. If the original topic query produces semantically relevant but vocabulary inappropriate results, the system searches simpler educational query variants and reranks the combined candidate pool. This makes the recommendation system more robust for difficult topics such as science, history, or politics, where direct Wikipedia results may be too advanced for beginner or intermediate readers.

G. Generation Component

The generator receives the student query, selected article context, vocabulary level, and new-word metadata. It returns a structured recommendation containing:

- article title
- short summary
- new vocabulary words and definitions
- confidence or difficulty rating
- rationale explaining why the article is a good next read

The deployed model is `Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf` from the Bartowski GGUF release of LLaMA 3.1 8B Instruct on Hugging Face. It is loaded through `llama-cpp-python` with GPU layer offloading, a 4096-token context window, and a low temperature generation configuration for stable structured output.

H. Story Generation

The story generator creates short personalized stories that are slightly above the student’s current reading level. It accepts topic, genre, and challenge level. The system then checks the generated story against the student’s known word list and reports known word ratio, unknown word ratio, new word count, target range, and whether the story falls inside the selected challenge range. The generator also cleans the model’s vocabulary list so that displayed challenge words actually appear in the story and are useful for learning.

I. Pretrained Instruction Tuned LLaMA Deployment

The generative layer uses a pretrained and instruction tuned LLaMA 3.1 8B model in GGUF format rather than a newly trained QLoRA adapter. The model already contains pretrained weights and instruction-following behavior, so the project did not need to train epochs or save LoRA adapter weights to produce recommendations. At runtime, the model is loaded locally through llama-cpp-python, which allows GPU layer offloading and efficient quantized inference.

TABLE IV
GENERATOR CONFIGURATION

Component	Value
Generator family	LLaMA 3.1 8B Instruct
Runtime file	Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf
Release source	Bartowski GGUF release
Runtime library	llama-cpp-python
Context window	4096 tokens
GPU offload	n_gpu_layers = -1
Purpose	Recommendations and story generation

This generator is separate from the embedding model. LLaMA produces user-facing text, while the retrieval index uses all-MiniLM-L6-v2 sentence-transformer embeddings. The Hugging Face token was used during setup to access or download gated model resources, but the deployed application loads the local GGUF file from the project model directory.

J. Multi-GPU Strategy

The workstation uses two NVIDIA GPUs: a Tesla P40 and a Tesla P41, each with approximately 24 GB of VRAM. The system verifies that both GPUs are detected and records GPU utilization during data processing, indexing, and model execution. The multi-GPU strategy uses explicit CUDA visibility and model/device placement to prevent accidental CPU-only execution. For LLaMA inference through llama-cpp-python, the model supports GPU layer offloading and optional tensor splitting. Because the final deployment uses a pretrained GGUF model rather than project-specific adapter training, multi-GPU evidence is reported through inference, indexing, and pipeline execution logs rather than QLoRA optimizer state partitioning. The GPUs are split 50/50 through tensor split 0.5 and 0.5.

VI. EXPERIMENTS AND RESULTS

A. Experimental Setup

Experiments were conducted on the ThinkStation workstation using the local project repository, SQLite application database, FAISS vector index, and local LLaMA model. The Wikipedia pipeline was evaluated on the 1,000,000 article download and 628,000 article cleaned/tokenized corpus. The web interface was tested with beginner, intermediate, and advanced student profiles.

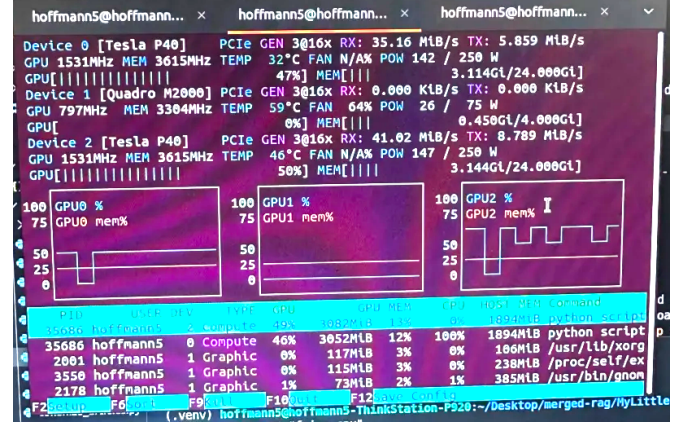


Fig. 3. GPU utilization screenshot showing both GPUs detected and under load during model or pipeline execution.

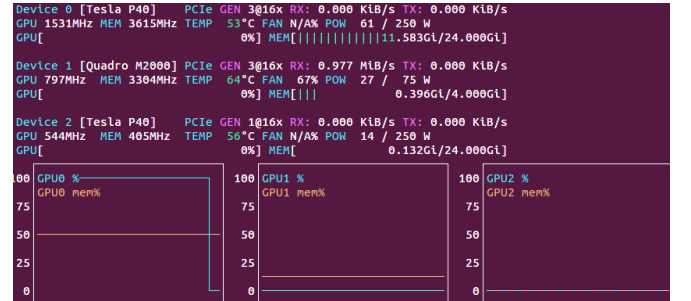


Fig. 4. System memory and process monitor during article statistics generation.

B. Corpus Results

The data pipeline successfully processed substantially more than the minimum 50,000 required articles. The final cleaned corpus contains approximately 628,000 usable articles, giving the recommender broad topic coverage. The exact final file names are outputs/article_stats_1m.jsonl, data/processed/wiki_1m_clean.parquet, data/processed/article_1m_tokenized.parquet, and the FAISS index directory size is approximately 5 GB.

TABLE V
PROCESSED ARTICLES AVAILABLE FOR EACH CHALLENGE SETTING

Level	Candidate Articles	Coverage Window
Low	628,652	95–98%
Medium	628,652	90–97%
High	628,652	85–93%

C. Retrieval Evaluation

Retrieval quality was evaluated using a fixed set of topic queries and expected relevant article labels. The evaluation reports Precision@k, Recall@k, and Mean Reciprocal Rank (MRR) for $k \in \{5, 10, 20\}$. Table VI provides the final results.

TABLE VI
RETRIEVAL METRICS

Metric	@5	@10	@20
Precision@k	.2533	.2067	.1600
Recall@k	.2533	.4133	.6400
MRR			.5534

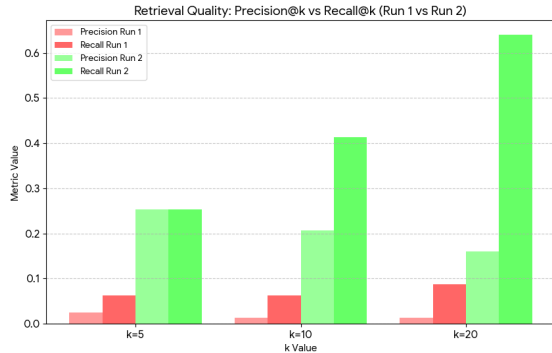


Fig. 5. Retrieval quality graph comparing Precision@k and Recall@k for $k = 5, 10$, and 20 .

D. Retrieval Ablation Study

The most important design decision in My Little Library is the addition of vocabulary aware reranking after semantic FAISS retrieval. Since the deployed system uses a pretrained instruction tuned LLaMA model rather than newly trained LoRA adapters, this ablation is the clearest implemented experiment for showing that a model or pipeline choice changed system behavior. The choice compares three configurations: semantic-only retrieval, semantic retrieval followed by a hard coverage filter, and the final weighted reranker that combines semantic relevance with vocabulary fit.

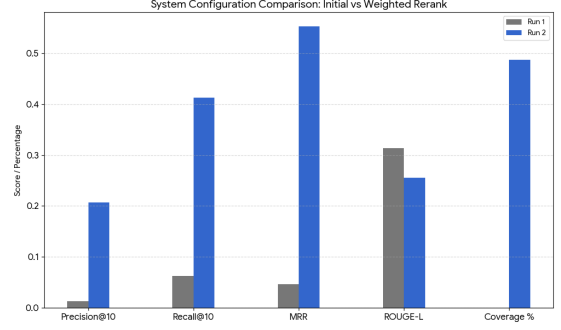


Fig. 6. Ablation graph comparing semantic-only FAISS retrieval, hard coverage filtering, and the final vocabulary-aware weighted reranker.

The expected outcome is not necessarily that the vocabulary aware configuration maximizes raw semantic precision. Instead, the final reranker should increase the percentage of displayed recommendations inside the target coverage window while preserving acceptable semantic relevance. This directly tests the project hypothesis: useful reading recommendations must be both topically relevant and vocabulary appropriate.

E. Vocabulary Awareness Evaluation

Vocabulary awareness measures how often recommended articles fall inside the target coverage window for the student’s level. This is separate from semantic relevance: a recommendation can be topically relevant but still too difficult. The final evaluation computes the percentage of displayed recommendations whose known-word coverage falls within the target range. Results should be reported both overall and separately for beginner, intermediate, and advanced profiles so that the demo narrative matches the three required student profiles.

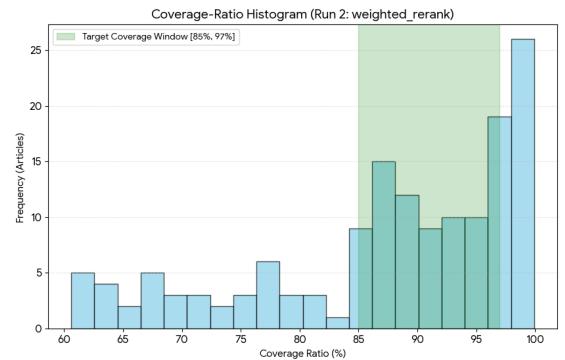


Fig. 7. Coverage-ratio histogram for recommended articles with the target coverage windows shaded.

F. Generation Evaluation

Generated summaries were compared against article abstracts using ROUGE and BERTScore. BERTScore is computed per generated-reference pair and then averaged across

the test set; therefore, the report should include aggregate precision, recall, and F1 values, plus optional per-level breakdowns.

TABLE VII
GENERATION QUALITY METRICS

Metric	Score
ROUGE-1	0.3035
ROUGE-2	0.1970
ROUGE-L	0.2550
BERTScore Precision	.7656
BERTScore Recall	.6997
BERTScore F1	.7308
Valid structured output rate	100%

G. Generator Evaluation

The deployed generator is a pretrained/instruction-tuned LLaMA model, so the evaluation compares the structured LLaMA output against a non-generative baseline such as retrieval-only metadata or template-based summaries. This keeps the evaluation aligned with the implemented system without incorrectly claiming project-specific QLoRA adapter training.

H. Evaluation and System Analytics

The project generated programmatic graphs for retrieval evaluation, generation evaluation, and coverage analysis. Because the deployed generator is a pretrained/instruction-tuned GGUF model rather than a project specific QLoRA adapter, training-loss and optimizer-step plots are not reported as adapter-training results. Instead, the analytics emphasize end-to-end inference behavior, retrieval quality, vocabulary awareness, and system resource use.

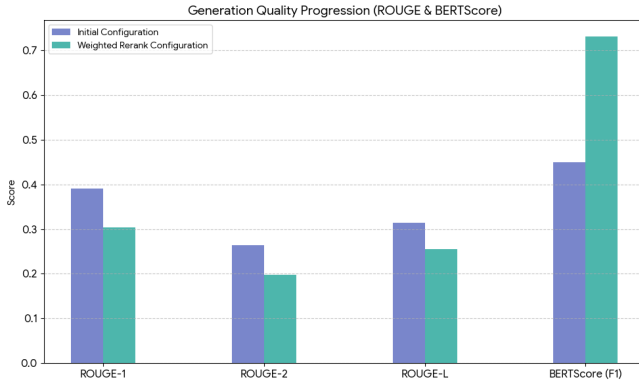


Fig. 8. Evaluation quality trend over evaluation batches or system configurations. Showing generation quality progression rather than adapter training loss.

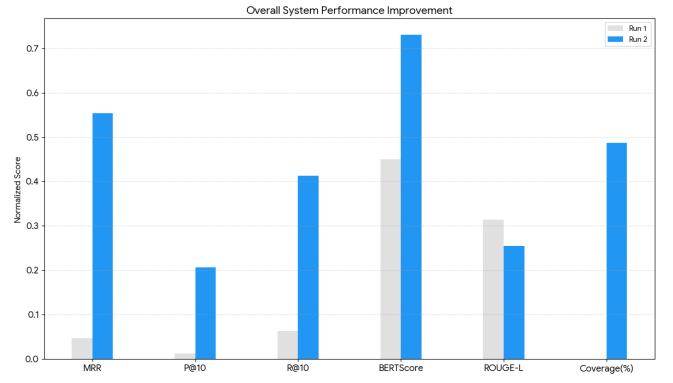


Fig. 9. ROUGE, BERTScore, retrieval quality, or vocabulary-awareness metrics across evaluation batches or system configurations.

I. System Integration and Demo

The integrated system runs through a Flask web interface. It accepts a student profile, displays current reading level, retrieves book suggestions, generates personalized stories, saves generated stories to the library, and updates the user experience based on profile state.

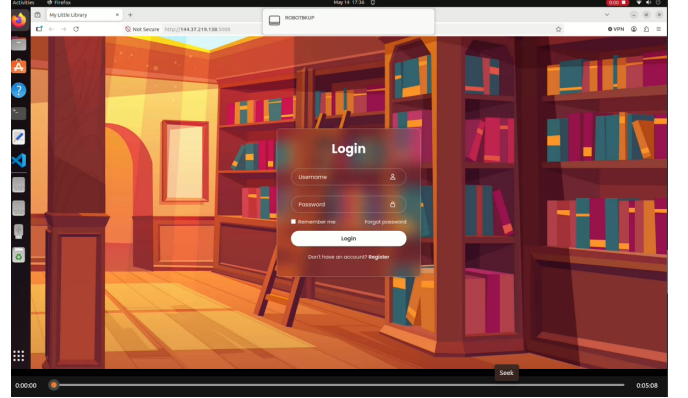


Fig. 10. Web application demo showing the starting page of the application. Link to the demo is in GitHub repository: <https://github.com/KellyLFrear/MyLittleLibrary>

J. Three-Profile Demo Results

The live demo was designed around three student profiles: beginner, intermediate, and advanced. Each profile uses a different vocabulary state and therefore should receive different article recommendations or story difficulty levels for the same general topic. Table VIII demonstrates the results from the recommendation generated output taking into account the users reading level, current known word list, current library, and personalized interests. The data is an approximation since articles vary in range per chunk and total word count affecting coverage and new words.

TABLE VIII
DEMO RESULTS BY STUDENT PROFILE

Profile	Topic	Coverage	New Words	Result
Beginner	Books I might like	67.66%	268	7th grade
Intermediate	Books I might like	85.7%	115	9th grade
Advanced	Books I might like	86.3%	124	12th grade

The values in Table VIII are the measured coverage values from the generated recommendation outputs, not the global candidate filter thresholds. Because recommendation chunks vary in length and difficulty, the measured coverage can fall outside the ideal target range during open-ended generation.

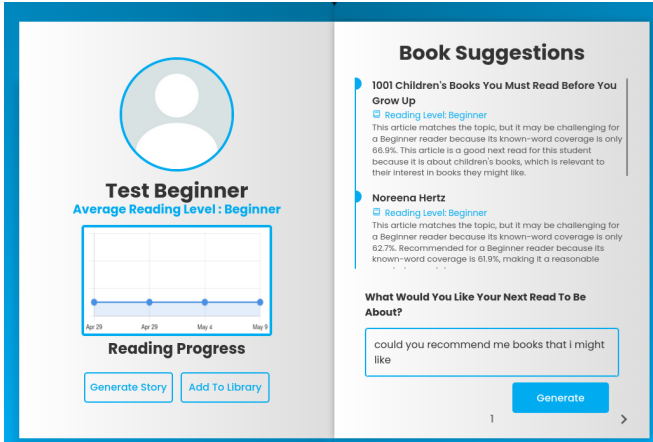


Fig. 11. Screenshot of figure showing recommendation behavior for beginner profile with explanation and reading level rating for each article.

K. Observed Performance Bottleneck

After moving to the 1,000,000-article source and larger FAISS index, the web application became slower to load and generate recommendations. The main causes are likely: increased FAISS index size, larger `chunks.pkl` meta-data loading, GPU transfer time for the FAISS index, first-request initialization of the sentence-transformer embedder, first-request initialization of the cross-encoder reranker, first-request LLaMA model loading, and reranking up to 200 broad candidates with a cross-encoder. The Flask server is also configured with `threaded=False`, which prevents concurrent request handling during long generation calls.

Recommended optimizations include loading the RAG pipeline at application startup rather than on the first user request, reducing `top_broad` from 200 to a smaller value for the live demo, caching query embeddings, disabling the cross-encoder for profile-only retrieval, using FAISS approximate search for very large indexes, moving metadata from a large pickle file into SQLite or memory-mapped storage, and enabling threaded or production WSGI serving for the frontend.

VII. DISCUSSION

A. What Worked

The system successfully integrates corpus processing, vocabulary analysis, dense retrieval, RAG generation, database-

backed profiles, and a live frontend. The largest practical achievement is that the system moved beyond a small prototype and processed a large Wikipedia corpus of approximately 628,000 usable articles. This gives the recommender enough topical coverage to respond to different student interests.

The vocabulary-aware reranking approach also worked well conceptually and is now evaluated through an explicit retrieval ablation. Semantic search alone can retrieve articles that match a topic but are too hard for a student. By adding coverage-based scoring, the recommender is better aligned with the educational objective: selecting a text that is understandable while still introducing useful new words.

The LLaMA-based generator improved the user-facing explanation layer. Instead of showing only article titles and raw scores, the system can explain why an article was recommended, summarize its content, define new vocabulary, and generate personalized stories.

B. What Did Not Work Initially

Several implementation challenges occurred during development. The data pipeline had to be scaled from small samples to hundreds of thousands of records. Vocabulary lists initially risked being nested across levels, which would have made intermediate and advanced lists include lower-level words. This was corrected by constructing level-specific lists. Frontend layout issues also appeared after interface updates, requiring CSS and page-structure fixes.

GPU configuration was another challenge. The workstation had to be configured so that CUDA, llama.cpp, FAISS, and PyTorch-based components used the intended GPUs rather than silently falling back to CPU. Our team collected screenshots and logs showing GPU utilization to verify that acceleration was active. However it is during testing with the larger data pipeline overheated the GPU causing cuda error 98 forcing us to fallback to single GPU usage.

[illegible]

Fig. 12. Screenshot of Cuda error trying to run evaluate rag script with both GPUs

C. Limitations

The current implementation has several limitations. First, recommendation quality depends on the accuracy of the known

word lists. A student’s true vocabulary may differ from the assumed grade-level preset. Second, Wikipedia articles can be uneven in quality, reading difficulty, and topical representation. Third, the current vector index and metadata loading strategy can become slow as the index grows. Fourth, exact FAISS¹ search is simple and accurate but may not be the best long-₂ term choice for million-scale chunk retrieval.

The deployed generator uses a pretrained/instruction-tuned³ GGUF model rather than a newly trained project-specific QLoRA adapter. The working system still demonstrates local⁴ LLaMA generation, RAG recommendations, vocabulary-aware reranking, GPU-accelerated execution, multi-GPU utilization⁵ evidence, and a system-level retrieval ablation. This design⁶ choice allowed us to focus on the end-to-end recommender, pipeline and reliable integration with the web application.

D. Ethical Considerations

Vocabulary aware educational systems must avoid treating reading level as a fixed measure of intelligence. A reading profile should be interpreted as a temporary support signal,¹ not a label of student ability. The system should let students² grow, override recommendations, and explore topics above,³ their estimated level when they choose.

There are also dataset concerns. Wikipedia coverage is not evenly distributed across cultures, regions, languages,⁵ and topics. Frequency lists and grade-band vocabulary lists may encode assumptions about standard English, academic language, and student background. Future versions should include teacher review, student feedback, and more diverse reading sources.

VIII. CONCLUSION AND FUTURE WORK

My Little Library implements an intelligent vocabulary recommender that combines large-scale corpus processing, vocabulary analysis, dense retrieval, RAG generation, local LLaMA inference, and a web-based student interface. The system processed approximately 1,000,000 downloaded Wikipedia articles into approximately 628,000 cleaned and tokenized articles, built vocabulary metadata for multiple reading levels, indexed article chunks in FAISS, and generated article recommendations and stories through a LLaMA-backed pipeline. The end-to-end demo supports student profiles, topic-based retrieval, vocabulary-aware filtering, generated explanations, and reading progress.

Future work would be focusing on improving latency on the 1,000,000 article index, adding approximate FAISS search, expanding evaluation queries, collecting human evaluation results, improving personalized vocabulary estimation, p-value testing, more detailed latency measurements, additional programmatic analytics graphs, and adding teacher facing controls. For current improvements many are related to features such as multigpu usage, coverage ratios closer to expected outcomes, and the development of a cloud based server application.

APPENDIX A COMMANDS AND REPRODUCIBILITY NOTES

Listing 1. Pipeline commands.

```
python scripts/download_wiki.py --sample-size 1000000 --
  output data/raw/wiki_lm.parquet
python scripts/preprocess_wiki.py --input data/raw/wiki_lm.
  parquet --output data/processed/wiki_clean.parquet
python scripts/tokenize_articles.py --input data/processed/
  wiki_clean.parquet --output data/processed/
  wiki_tokenized.parquet
python scripts/analyze_articles.py --input data/processed/
  wiki_tokenized.parquet --output outputs/
  article_stats_lm.jsonl
python scripts/build_index.py --input outputs/
  article_stats_lm.jsonl --output data/faiss_index
python scripts/evaluate_rag.py --queries data/eval_queries.
  json --index data/faiss_index
python app.py
```

APPENDIX B ABLATION AND EVALUATION COMMANDS

Listing 2. Ablation commands.

```
python scripts/evaluate_rag.py --mode semantic_only --
  queries data/eval_queries.json
python scripts/evaluate_rag.py --mode coverage_filter --
  queries data/eval_queries.json
python scripts/evaluate_rag.py --mode weighted_rerank --
  queries data/eval_queries.json
python scripts/evaluate_generation.py --metrics rouge
  bertscore --outputs outputs/generated_eval.jsonl
python scripts/significance_test.py --baseline outputs/
  baseline_eval.jsonl --system outputs/reranker_eval.
  jsonl
```

APPENDIX C SAMPLE OUTPUT

A. Json story generation output

Prompt for intermediate user wanting a story about space.
A Galactic Quest

As the spaceship hurtled through the galaxy, Captain Ford peered out the cockpit window at the adjacent stars. Their mission was to explore a mysterious planet on the other side of the cosmos. The applicant for the job had been an experienced astronaut, but they were blind to the perils of space travel. Ford’s father, Paul Hoffmann, had been a renowned physicist-computer scientist, but his love of composition and rhythm had led him to ignore the dangers of space exploration, forever being lost to the abyss. The planet’s atmosphere was composed of sodium and oxygen, elements that seemed desirable for life, but Ford knew better. They had studied the episode of the planet’s exodus, and it was clear that it was a terrible place to be. As they approached the planet, the flow of fuel to the engines became intermittent, and Ford’s heart sank. They had to rely on their keen imagination to navigate the hangar of asteroids and avoid a loud explosion. The satisfaction of completing their mission was tainted by the thought of the plaintiff astronaut’s family, who would never know what had happened to their loved one. Ford poured over the procedure for the mission, trying to find a way to make up for the lack of fuel. They had to use their trust in their training to overcome the struggle of finding a solution. The overview of the planet’s

surface was a blur of red and orange, and Ford’s puppy, EE, was getting anxious. They had to swap the fuel to the backup tank and pour in the remaining fuel from the reserve. The mug of coffee on their console was cold, but it was a reminder of the comforts of home. Ford’s heart swelled with trust in her abilities as she completed the mission and headed back towards the galaxy knowing their father would be proud.

B. JSON story recommendation output

Intermediate user asking for articles about space.

Isaac Asimov’s Guide to Earth and Space Reading Level: Intermediate

This article is a good next read for the student because it is about space, which aligns with their interest, and it is written at an intermediate level, which matches their vocabulary level.

A Brief History of Time Reading Level: Intermediate

This article is a good next read for the student because it is a book about space and cosmology, which aligns with their interest and reading level.

The Conquest of Space (Lasser book) Reading Level: Intermediate

This article is a good next read for this student because it aligns with their interest in space and is written at a level that matches their vocabulary skills.

APPENDIX D

INDIVIDUAL CONTRIBUTION SUMMARY

Kelly Frear: During this semester, I was in charge of creating the PowerPoints and developing the front end portions of the project. I created the entire login page along with the entire front-end “book” page for the rest of the project. In addition to the “cosmetics” of the project, I also wrote the scripts that’d grab the Wikipedia articles and cleaned them in order for them to be used.

George Muniz: For this project, I focused on model integration, recommendation and story generation optimization, SQLite schema design, vocabulary pipeline and preprocessing scripts, FAISS index preparation for 50k, 100k, and 1m article runs, portions of the embedding workflow using all-MiniLM-L6-v2, report writing, and system architecture diagrams.

Gavin Rosander: I mostly handled the backend development, and generating visual graphs for the final report. I designed the RAG agent, story generation service, designed the SQLite database for users, mocked our test data, and figured out how to distribute the workload across both GPU’s. Evaluated the RAG agent to generate graphs for the final report.

APPENDIX E

USE OF AI ASSISTANCE

Throughout the project, AI assistance was used as a development support tool for planning, debugging, documentation, and refinement. We used ChatGPT/Gemini to help identify missing rubric requirements, improve implementation plans, and clarify technical decisions. AI assistance was not used as a replacement for building or testing the system; rather, it

helped us reason through design choices, debug errors, and convert project progress into clearer technical documentation.

One major use of AI assistance was project planning and architecture design. We asked for guidance on how the full My Little Library system should be structured, including how the offline Wikipedia data pipeline, vocabulary analysis module, FAISS vector database, RAG pipeline, LLaMA generator, SQLite database, and Flask web interface should connect. This helped us decide the system was a modular layered monolith with an offline data processing pipeline and an online RAG inference pipeline. In several cases, AI assistance helped rewrite or modify code so that it better matched the project requirements, such as improving evaluation script arguments, fixing BERTScore evaluation logic, and updating `requirements.txt` with missing dependencies. Debugging was another important use case. We used AI assistance to interpret terminal errors, LaTeX compilation errors, GPU configuration issues, CUDA problems, Git commands, Flask/frontend behavior, and report formatting issues. AI assistance helped explain why tables floated to unexpected positions in IEEE format, why the web application slowed after moving to the larger one million article index, and how to verify GPU usage through CUDA tools. AI assistance also helped with evaluation design. We asked for guidance on how to align the implemented system with the project rubric, especially after deciding to use a pretrained/instruction-tuned LLaMA 3.1 8B GGUF model instead of training new QLoRA adapter weights. AI assistance helped reframe the evaluation around the actual implemented system by emphasizing retrieval quality, vocabulary awareness, semantic-only versus vocabulary-aware reranking ablation, ROUGE, BERTScore, structured-output validity, coverage distributions, GPU utilization, and latency behavior. This helped us complete an honest evaluation of the working system.

Finally, AI assistance was used in report writing and revision. We asked for help converting an earlier report template into an IEEE style technical report that matched the CS 322 rubric. AI assistance helped identify unclear wording, remove unfinished or informal language, improve consistency, and ensure that the report accurately described the current code base.

Overall, AI assistance supported the project as a technical advisor, debugging partner, and documentation editor. The development, implementation, testing, screenshots, evaluation outputs, and project decisions were still completed and verified by us. The use of AI improved our ability to organize the project, resolve implementation issues, and communicate the system clearly in the final report.

REFERENCES

- [1] I. S. P. Nation, *Learning Vocabulary in Another Language*. Cambridge, U.K.: Cambridge University Press, 2001.
- [2] M. West, *A General Service List of English Words*. London, U.K.: Longman, 1953.
- [3] C. Browne, B. Culligan, and J. Phillips, “The New General Service List,” 2013.

- [4] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas for Navy enlisted personnel," Naval Technical Training Command, Research Branch Report 8-75, 1975.
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.
- [6] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
- [7] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459-9474.
- [8] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023.
- [10] M. Shoybi *et al.*, "Megatron-LM: Training multi-billion parameter language models using model parallelism," arXiv preprint arXiv:1909.08053, 2019.
- [11] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.
- [12] S. Li *et al.*, "PyTorch Distributed: Experiences on accelerating data parallel training," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3005-3018, 2020.
- [13] S. Gugger *et al.*, "Accelerate: Training and inference at scale made simple, efficient and adaptable," 2022.
- [14] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74-81.
- [15] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. ICLR*, 2020.
- [16] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38-45.
- [17] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.